Chapter 2 Python for Natural Language Processing

String Representation

```
new_string = "This is a String" # storing a string
```

```
print('ID:', id(new_string)) # shows the object identifier (address)
```

```
print('Type:', type(new_string)) # shows the object type
```

```
print('Value:', new_string) # shows the object value
```

```
print(new_string)
```

```
new_string
```

More String Representations

simple string

```
simple_string = 'Hello!' + " I'm a simple string"
print(simple_string)
```

multi-line string, note the \n (newline) escape character automatically created

```
multi_line_string = """Hello I'm
a multi-line
string!"""
```

```
multi_line_string
```

```
print(multi_line_string)
```

Escape Sequences

Let's now look at ways to represent escape sequences and raw strings in their native format, without any escape sequences.

```
# Normal string with escape sequences leading to a wrong file path!
escaped_string = "C:\the_folder\new_dir\file.txt"
```

```
print(escaped_string) # will cause errors if we try to open a file here
```

```
# raw string keeping the backslashes in its normal form
raw_string = r'C:\the_folder\new_dir\file.txt'
```

```
print(raw_string)
```

ASCII vs Unicode

- **ASCII (American Standard Code for Information Interchange):**
    - **Introduced:** 1963
    - **Character Set Size:** 7-bit encoding, supporting 128 characters, extended to 8-bit in 1981
    - **Scope:** Primarily designed for English characters and control codes
    - **Limitations:** Limited support for non-English characters and symbols
- **Unicode:**
    - **Introduced:** 1991
    - **Character Set Size:** Variable-length encoding (e.g., UTF-8, UTF-16) supporting over 1 million characters
    - **Scope:** Designed to represent characters from all writing systems worldwide
    - **Advantages:** Comprehensive support for global languages, symbols, and emojis

ASCII vs Unicode (cont'd)

Let's look at ways to represent non-ASCII characters leveraging Unicode, which can also be used to represent symbols like emojis.

\u followed by 4-digit hexadecimal numbers (in Python).

```
# unicode string literals
string_with_unicode = 'H\u0061llo!'
```

```
print(string_with_unicode)
```

```
# unicode string literals
string_with_unicode = 'H\u00e8llo!'
```

```
print(string_with_unicode)
```

More Unicode

\U followed by 8-digit hexadecimal numbers (in Python).

```
more_unicode = 'I love Pizza \U0001F355! \
Shall we book a cab \U0001F695 to get pizza?'
```

```
print(more_unicode)
```

```
print(string_with_unicode + '\n' + more_unicode)
```

```
' '.join([string_with_unicode, more_unicode])
```

More Fun with Emojis "Emojis - just doesn't count!"

Let's search for some website with emojis.
For example, https://en.wikipedia.org/wiki/Emoji

```
'Hello 😊' + ' and welcome ' + 'to Python 🐍!'
```

```
character = '😊'
code_point = ord(character)
```

```
print(code_point)
print(f"The Unicode code point of the character {character} \
is U+{code_point:04X}.")
```

```
character = '🐍'
code_point = ord(character)
```

```
print(code_point)
print(f"The Unicode code point of the character {character} \
is U+{code_point:04X}.")
```

String Operations and Methods

Basic Operations

```
'Hello \U0001F60A' + ' and welcome ' + 'to Python \U0001F40D!'
```

```
'Hello \U0001F60A' ' and welcome ' 'to Python \U0001F40D!'
```

```
s1 = 'Python \U0001F40D!'
```

```
'Hello \U0001F60A ' + s1
```

```
s2 = '\U0001F40DPython\U0001F40D'
```

Multiplication?!

```
s2*5
```

```
s1 + s2
```

```
(s1 + s2)*3
```

More String Operations

```
s3 = ('This ' 'is another way ' 'to concatenate ' 'several strings!')
```

```
s3
```

```
'way' in s3
```

```
'Python' in s3
```

```
len(s3)
```

Indexing and Slicing

```
s = 'PYTHON'
```

```
s, type(s)
```

```
s[0], s[1], s[2], s[3], s[4], s[5]
```

```
s[-1], s[-2], s[-3], s[-4], s[-5], s[-6]
```

```
for character in s:
    print(character)
```

```
for index in range(len(s)):
    print(index, s[index])
```

```
for index, character in enumerate(s):
    print('Character ->', character, 'has index->', index)
```

Indexing and Slicing (cont'd)

```
s[:]
```

```
s[1:4]
```

```
s[:3], s[3:]
```

```
s[-3:]
```

```
s[:3] + s[-3:]
```

Indexing and Slicing (cont'd)

```
s[::1] # no offset
```

```
s[::-1] # backwards/reverse
```

```
s[::2] # print every 2nd character in string
```

```
#s[0] = 'X' # immutable!!
```

```
print('Original String id:', id(s))
```

```
# creates a new string
s = 'X' + s[1:]
```

```
print(s)
print('New String id:', id(s))
```

Methods

```
s = 'python is great'
```

Case Conversions

```
s.capitalize()
```

```
s.upper()
```

```
s.title()
```

String Replace

```
s.replace('python', 'NLP')
```

Numeric Checks

```
'12345'.isdecimal()
```

```
'apollo11'.isdecimal()
```

Alphabet Checks

```
'python'.isalpha()
```

```
'number1'.isalpha()
```

Alphanumeric Checks

```
'total'.isalnum()
```

```
'abc123'.isalnum()
```

```
'1+1'.isalnum()
```

String Splitting and Joining

```
s = 'I,am,a,comma,separated,string'
s.split(',')
```

```
' '.join(s.split(','))
```

Basic String Stripping

```
s = ' I am surrounded by spaces '
s
```

```
print(s)
```

```
s.strip()
```

Some More Combinations

```
sentences = 'Python is great. NLP is also good.'
sentences.split('.')
```

```
print('\n'.join(sentences.split('.')))
```

```
[sentence.strip()
                for sentence in sentences.split('.') if sentence]
```

```
print('\n'.join([sentence.strip()
                for sentence in sentences.split('.') if sentence]))
```

Formatting

Simple string formatting expressions

old style (since Python's inception)

```
'Hello %s' %('Python!')
```

```
'Hello %s %s' %('World!', 'How are you?')
```

Formatting expressions with different data types

old style (since Python's inception)

```
'We have %d %s containing %.2f gallons of %s' \
%(2, 'bottles', 2.5, 'milk')
```

```
'We have %d %s containing %.2f gallons of %s' \
%(5.21, 'jugs', 10.86763, 'juice')
```

Formatting strings using the format method

newer style (since Python 2.7)

```
'Hello {} {}, it is a great {} to meet you at {}' \
.format('Mr.', 'Jones', 'pleasure', 5)
```

```
'Hello {} {}, it is a great {} to meet you at {} o\'clock' \
.format('Sir', 'Arthur', 'honor', 9)
```

Alternative ways of using string format

```
'I have a {food_item} and a {drink_item} with me' \
.format(drink_item='soda', food_item='sandwich')
```

```
'The {animal} has the following attributes: {attributes}' \
.format(animal='dog', attributes=['lazy', 'loyal'])
```

f(ormatted)-string (since Python 3.6)

```
animal='dog'
attributes=['lazy', 'loyal']
```

First the two variations of strings with .format:

```
'The {animal} has the following attributes: {attributes}' \
.format(animal=animal, attributes=attributes)
```

```
'The {} has the following attributes: {}' \
.format(animal, attributes)
```

Then the f-string:

```
f'The {animal} has the following attributes: {attributes}'
```

```
f'The {animal} has the following attributes: {", ".join(attributes)}'
```

A date example:

```
from datetime import datetime
```

```
current_time = datetime.now()
```

```
current_time
```

```
formatted_date = f'{current_time:%Y-%m-%d}'
```

```
formatted_date
```

Additional Readings

Text Analytics with Python: A Practitioner's Guide to Natural Language Processing Paperback, 2019, by Dipanjan Sarkar

https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str

https://docs.python.org/3/library/datetime.html#strftime-and-strptime-format-codes